



SE 342 01 SOFTWARE VALIDATION AND TESTING

FINAL PROJECT

-EXAM SECURITY SYSTEM-

PROJECT MEMBERS

Burçak Çelt	2207006034
Ecem Nur Özer	220706038

Exam Security System – Project Overview

The Exam Security System is a web-based application designed to support secure and controlled exam-day operations. The main objective of the system is to verify student identity during exam entry, ensure seating plan compliance, and record any violations that may occur throughout the examination process. The system aims to reduce manual errors and strengthen exam integrity by combining rule-based validation with a simple image verification component.

The system supports three main user roles: Student, Proctor (Invigilator), and Exam Coordinator (Admin). Each role has clearly defined permissions and responsibilities within the system. While students are passive participants in the process, proctors are responsible for check-in operations and violation logging, and administrators manage exam-related data such as exams, rooms, and seating plans.

System Requirements

This section defines the functional and non-functional requirements of the Exam Security System. These requirements form the basis for system design, implementation, and testing activities.

Functional Requirements

The system shall provide secure authentication and role-based access control for proctors and administrators. Users must only be able to access features permitted by their assigned roles.

The system shall allow administrators to create and manage exams, including exam date, time, and assigned rooms. For each exam, a seating plan must be defined using either seat codes or a row-column structure.

The system shall support student roster management by allowing basic student information to be entered or imported prior to the exam.

During the exam check-in process, the system shall allow proctors to capture or upload a student photo. This image shall be processed by the image verification component to determine whether the student matches the registered identity.

The system shall validate whether the student is seated in the correct seat according to the predefined seating plan. The result of both identity verification and seating compliance checks shall be recorded with a timestamp.

The system shall allow proctors to log exam violations by specifying a violation reason, optional notes, and optional evidence images.

The system shall provide basic reporting features, including lists of check-ins, identity mismatches, seating violations, and recorded incidents

Non-Functional Requirements

The system shall enforce security measures to prevent unauthorized access and ensure that sensitive exam and identity data is protected.

The system shall be designed to be usable in a real exam environment, allowing proctors to complete the check-in process with minimal steps and clear feedback.

The system shall provide reasonable performance, ensuring that the check-in workflow completes within an acceptable time frame to avoid delays during exam entry.

The system shall maintain data consistency and reliability, preventing duplicate check-ins and invalid data entries.

Business Rules and Validation Constraints

Each student shall be allowed to perform the check-in process only once per exam. Any attempt to perform a duplicate check-in must be rejected by the system.

Only users with the Proctor role shall be allowed to perform student check-ins and log violations. Administrative features such as exam and seating plan creation shall be restricted to users with the Admin role.

A violation record must include a valid violation reason. Records without a reason shall not be accepted by the system.

If the image verification component returns a mismatch result, the system shall flag the check-in and allow the proctor to record a violation if necessary.

If a student is detected to be seated in an incorrect seat, the system shall record the seating mismatch and mark the check-in accordingly.

Transition to Project Planning and Task Management

Based on the defined system requirements and business rules, the project development process was planned and managed using an issue-based task management approach. The identified requirements were mapped to implementation, design, and testing activities to ensure traceability and controlled progress throughout the project lifecycle.

Project Repository and Version Control

The Exam Security System project was developed and maintained using a centralized version control repository. All source code, documentation, diagrams, database scripts, and test-related files were stored in a single repository to ensure consistency, traceability, and reproducibility of the project.

The repository structure follows the required submission format and clearly separates analysis documents, system diagrams, database files, application source code, and testing artifacts. Version control was used throughout the development process to track changes, manage incremental progress, and maintain a clear development history.

The repository also serves as the primary reference for running the application, executing tests, and reviewing the implemented features. Detailed instructions for setup and execution are provided in the README file within the repository.

Repository Link:

<https://github.com/ecemozr/-Exam-Security-System>

Project Planning and Backlog Structure

After defining the system requirements and business rules, the development process of the Exam Security System was organized using a structured backlog. The project tasks were grouped into epics based on functional areas of the system. This approach helped ensure traceability between requirements, design decisions, implementation steps, and testing activities.

Epic 1 – Requirements & Analysis

This epic focuses on understanding the project scope and defining the core requirements of the system. Functional and non-functional requirements were analyzed based on the project scenario, and system constraints were documented as business rules. These outputs served as the foundation for all subsequent design, implementation, and testing activities.

Epic 2 – System Design & Diagrams

The System Design & Diagrams epic covers the visual modeling of the system structure and workflows. UML diagrams were prepared to represent user interactions, database structure, and system behavior during the exam check-in

process. These diagrams were created in alignment with the defined requirements to ensure consistency between analysis and design.

As a bonus activity, an activity diagram was also prepared to illustrate the overall exam check-in flow with decision points for identity verification, seat compliance, and violation handling.

Epic 3 – Project Planning & Task Management

This epic addresses the organization and management of project work. All project activities were divided into stories and tasks, assigned to team members, and planned across sprints. The sprint board was used to track progress and maintain a clear overview of completed and ongoing work throughout the development process.

Epic 4 – Database Implementation

The Database Implementation epic focuses on designing and implementing the database layer of the system. Based on the ER diagram, database tables, relationships, and constraints were created. Dummy data was inserted to support development, testing, and demonstration of system functionality.

Epic 5 – Core System Implementation

This epic covers the implementation of the main system features. Authentication and role-based access control were implemented to enforce permissions. Exam creation, room management, seating plan definition, and the student check-in workflow were developed according to the system requirements. Violation logging and basic reporting functionalities were also included within this epic.

Epic 6 – ML Integration

The ML Integration epic focuses on integrating a simple image verification component into the system. A wrapper service was implemented to handle image input and return a match or no-match decision. The purpose of this epic is not to develop a complex machine learning model, but to ensure correct integration, validation, and decision handling within the system workflow.

Epic 7 – Testing & Validation

This epic addresses the testing and validation activities of the project. Functional, negative, and edge case test scenarios were documented in a structured test cases document. In addition, unit tests were implemented for critical system logic, including seating compliance, duplicate check-in prevention, role-based access rules, and the ML wrapper service using mock inputs.

Epic 8 – Documentation & Finalization

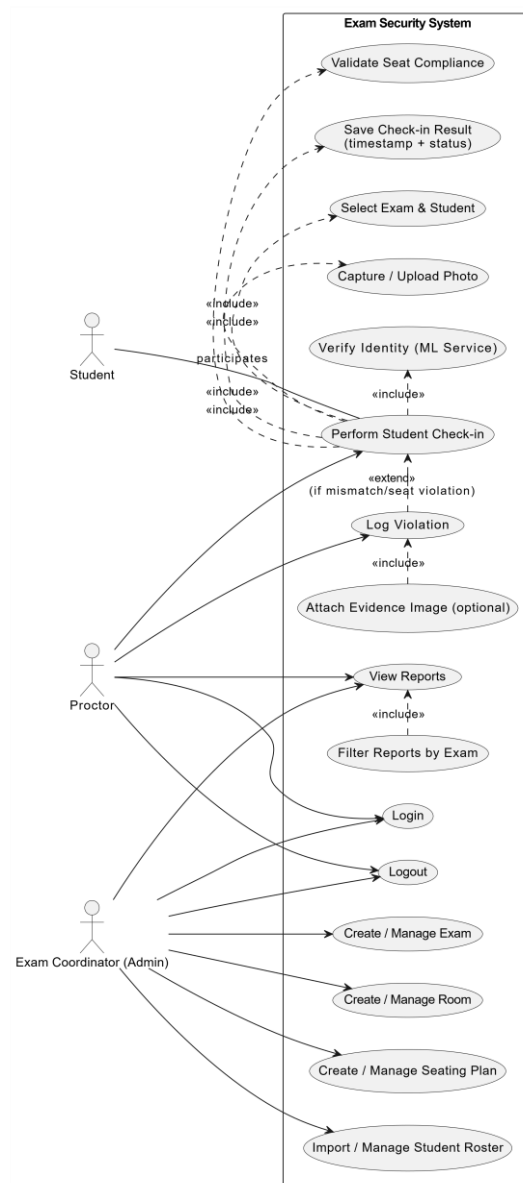
The final epic focuses on preparing the project for submission. System usage instructions, test execution steps, and a demo scenario were documented. A final review was conducted to ensure consistency with the project rubric and completeness of all required deliverables.

System Diagrams

This section presents the UML diagrams used to model the structure and behavior of the Exam Security System. The diagrams were designed based on the defined requirements and support a clear understanding of system interactions, data relationships, and operational workflows.

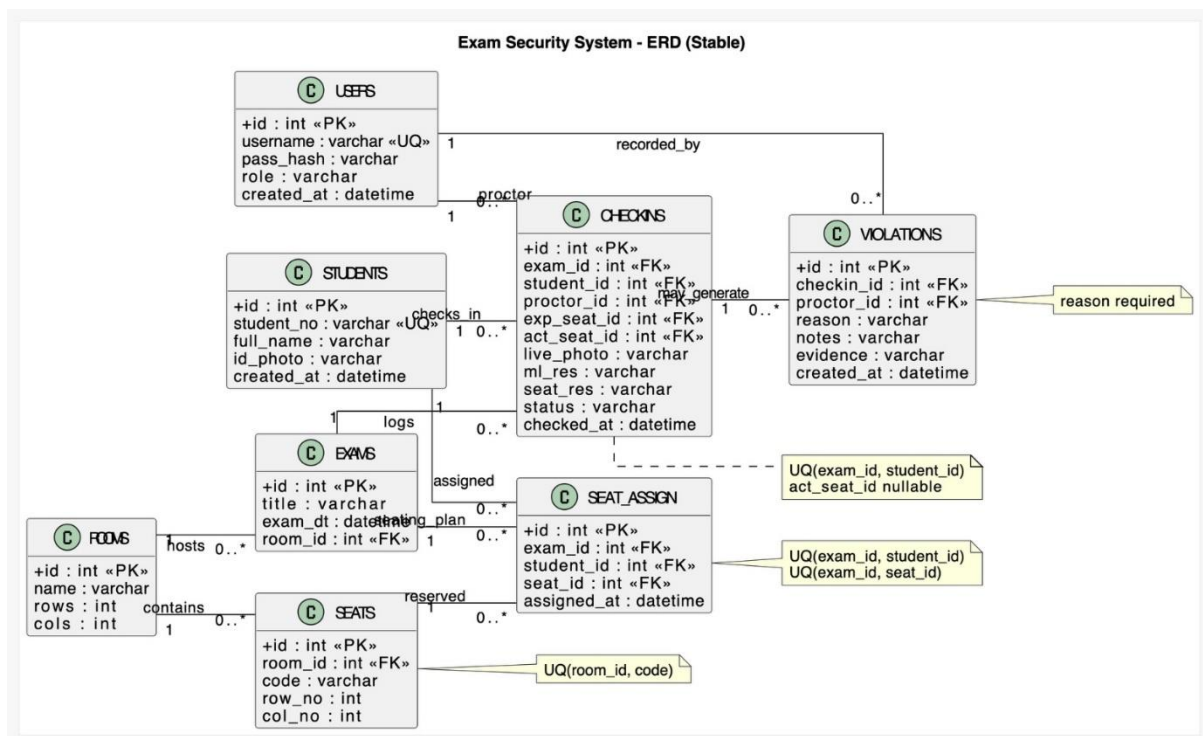
Use Case Diagram

The use case diagram illustrates the interactions between system actors and the Exam Security System. It defines the responsibilities of each actor and the system functionalities they can access. This diagram helps visualize the scope of the system from a user perspective.



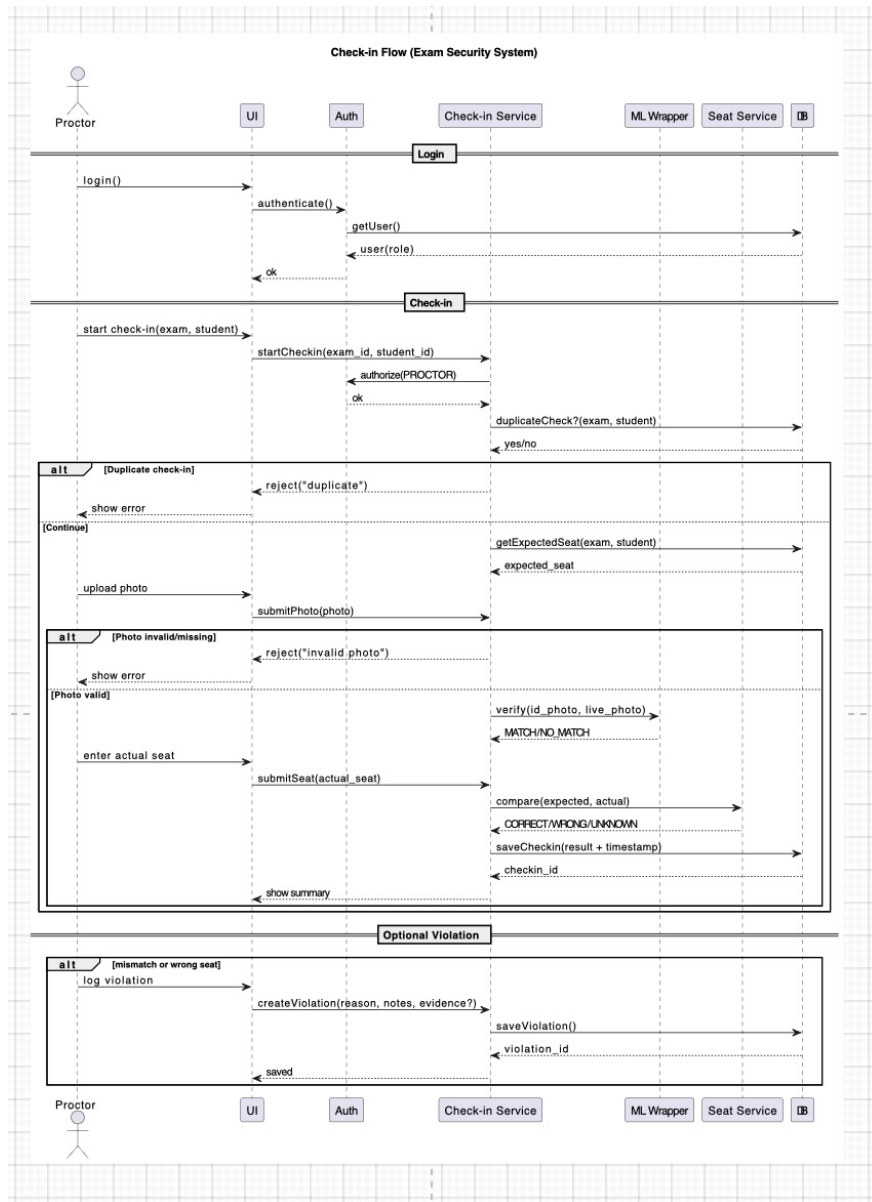
Entity Relationship Diagram (ERD)

The ERD represents the database structure of the system, including entities, attributes, and relationships. It ensures data consistency and supports the enforcement of business rules such as unique check-ins and role-based access.



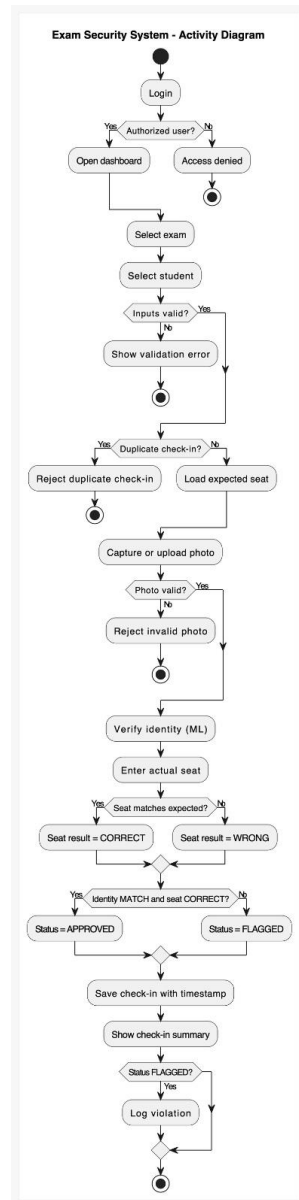
Sequence Diagram

The sequence diagram models the exam check-in workflow in detail. It shows the interaction between the proctor, system components, image verification service, and database during the identity verification and seat compliance process.



Activity Diagram

The activity diagram provides a high-level view of the exam check-in process. It highlights the flow of actions and decision points, including identity verification results, seat compliance checks, and violation handling.



JIRA Sprint Planning and Progress Tracking

Project development was managed using an issue-based sprint planning approach. After defining the system requirements and overall scope, project tasks were organized into epics and stories representing analysis, design, implementation, database development, testing, and documentation activities. This structure enabled clear tracking of progress and ensured alignment between requirements and implementation.

The backlog was created by decomposing high-level system requirements into manageable stories and tasks. Each story was associated with a specific epic, allowing related tasks to be grouped logically. Tasks were then assigned to team members and distributed across sprints based on priority and dependency.

During the sprint execution phase, tasks progressed through standard workflow states such as *To Do*, *In Progress*, and *Done*. This visual representation allowed the team to monitor progress, identify bottlenecks, and ensure timely completion of critical components such as system diagrams, database schema, and core check-in functionality.

Sprint boards were updated regularly to reflect completed work and remaining tasks. At the end of the development process, the sprint board provided a clear overview of completed epics and demonstrated that all required deliverables were addressed. A screenshot of the final sprint board is included as evidence of project planning and task management.

Jira interface showing the "Exam Security System" project. The left sidebar contains navigation options: For you, Recent, Starred, Apps, Plans, Spaces, Recent, Exam Security System, Student Attendance S..., View all spaces, Filters, Dashboards, Teams, and Customise sidebar. The main content area displays a list of tasks under the "Exam Security System" space. The tasks are organized into a table with columns: Type, Key, Summary, Status, Comments, Sprint, and Assignee. The tasks are listed as follows:

Type	Key	Summary	Status	Comments	Sprint	Assignee
Requirement	ESS-1	Requirements & Analysis	DONE	Add comment		Ecem Özer
Requirement	ESS-6	Identify system requirements	DONE	Add comment		Ecem Özer
Requirement	ESS-7	Review final project PDF and scenario	DONE	Add comment		Burçak Çelt
Requirement	ESS-8	List core system functionalities	DONE	Add comment		Burçak Çelt
Requirement	ESS-9	Identify non-functional requirements (security, usability,...)	DONE	Add comment		Ecem Özer
Requirement	ESS-10	Define business rules	DONE	Add comment		Ecem Özer
Requirement	ESS-11	Define check-in validation rules	DONE	Add comment		Burçak Çelt
Requirement	ESS-12	Define role-based access rules	DONE	Add comment		Burçak Çelt
Requirement	ESS-13	Define violation and reporting rules	DONE	Add comment		Burçak Çelt
Requirement	ESS-15	System Design & Diagrams	DONE	Add comment		Ecem Özer
Requirement	ESS-16	Create use case diagram	DONE	Add comment		Ecem Özer
Requirement	ESS-18	Draw use case diagram	DONE	Add comment		Ecem Özer
Requirement	ESS-19	Review consistency with requirements	DONE	Add comment		Burçak Çelt
Requirement	ESS-17	Identify actors (Student, Proctor, Admin)	DONE	Add comment		Burçak Çelt

Buttons: + Create, Quickstart

Jira interface showing the "Exam Security System" project. The left sidebar contains navigation options: For you, Recent, Starred, Apps, Plans, Spaces, Recent, Exam Security System, Student Attendance S..., View all spaces, Filters, Dashboards, Teams, and Customise sidebar. The main content area displays a list of tasks under the "Exam Security System" space. The tasks are organized into a table with columns: Type, Key, Summary, Status, Comments, Sprint, and Assignee. The tasks are listed as follows:

Type	Key	Summary	Status	Comments	Sprint	Assignee
Requirement	ESS-20	Design database ERD	DONE	Add comment		Ecem Özer
Requirement	ESS-21	Identify database entities	DONE	Add comment		Burçak Çelt
Requirement	ESS-22	Define relationships and constraints	DONE	Add comment		Burçak Çelt
Requirement	ESS-23	Draw ERD	DONE	Add comment		Ecem Özer
Requirement	ESS-24	Design check-in sequence diagram	DONE	Add comment		Ecem Özer
Requirement	ESS-25	Identify sequence steps	DONE	Add comment		Burçak Çelt
Requirement	ESS-26	Draw sequence diagram	DONE	Add comment		Burçak Çelt
Requirement	ESS-27	Validate diagram with implementation plan	DONE	Add comment		Ecem Özer
Requirement	ESS-28	Create activity diagram	DONE	Add comment		Ecem Özer
Requirement	ESS-29	Identify main activities in the check-in process	DONE	Add comment		Burçak Çelt
Requirement	ESS-30	Add decision nodes (face match, correct seat, duplicate ...)	DONE	Add comment		Burçak Çelt
Requirement	ESS-31	Model alternative flows for mismatches and violations	DONE	Add comment		Ecem Özer
Requirement	ESS-32	Review diagram for clarity and completeness	DONE	Add comment		Ecem Özer
Requirement	ESS-33	JIRA Planning & Project Management	DONE	Add comment		Burçak Çelt
Requirement	ESS-34	Designing sprint board	DONE	Add comment		Burçak Çelt

Buttons: + Create, Quickstart

Jira

Search

+ Create Upgrade

Spaces

Exam Security System

Summary Backlog Board Timeline List Forms Development Pages

Search list Filter

Group

	Type	Key	Summary	Status	Comments	Sprint	Assignee	
☐		ESS-38	Organize sprint board	DONE	Add comment		Burçak Çelt	
☐		ESS-41	Take sprint board screenshot for submission	DONE	Add comment		Ecem Özer	
☐		ESS-40	Update task statuses	DONE	Add comment		Ecem Özer	
☐		ESS-39	Move tasks to sprint	DONE	Add comment		Burçak Çelt	
☐		ESS-34	Create project backlog	DONE	Add comment		Burçak Çelt	
☐		ESS-35	Create epics and stories	DONE	Add comment		Ecem Özer	
☐		ESS-36	Assign tasks to team members	DONE	Add comment		Ecem Özer	
☐		ESS-37	Estimate task durations	DONE	Add comment		Burçak Çelt	
☐		ESS-42	Database Implementation	DONE	Add comment		Burçak Çelt	
☐		ESS-43	Create database schema	DONE	Add comment		Ecem Özer	
☐		ESS-44	Create tables	DONE	Add comment		Ecem Özer	
☒		ESS-45	Add primary and foreign keys	DONE	Add comment		Burçak Çelt	
☐		ESS-46	Add unique constraints	DONE	Add comment		Burçak Çelt	
☐		ESS-47	Insert dummy data	DONE	Add comment		Ecem Özer	
☐		ESS-48	Create sample users (admin, proctor)	DONE	Add comment		Ecem Özer	

+ Create

Quickstart

Jira

Search

+ Create Upgrade

Spaces

Exam Security System

Summary Backlog Board Timeline List Forms Development Pages

Search list Filter

Group

	Type	Key	Summary	Status	Comments	Sprint	Assignee	
☐		ESS-48	Create sample users (admin, proctor)	DONE	Add comment		Ecem Özer	
☐		ESS-49	Create sample exams and seating plans	DONE	Add comment		Ecem Özer	
☐		ESS-50	Insert sample students	DONE	Add comment		Ecem Özer	
☐		ESS-51	Core System Implementation	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-65	Implement violation logging	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-68	Link violation to student and exam	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-67	Save violation details	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-66	Create violation form	IN PROGRESS	Add comment		Burçak Çelt	
☒		ESS-52	Implement authentication and roles	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-53	Implement login functionality	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-55	Test permission rules	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-54	Restrict access based on roles	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-56	Implement exam and seating management	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-57	Implement exam creation	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-58	Implement room and seating plan creation	IN PROGRESS	Add comment		Burçak Çelt	

+ Create

Quickstart

burcakcelt.atlassian.net

Jira

Search

+ Create Upgrade

Spaces

Exam Security System

Summary Backlog Board Timeline List Forms Development Pages

Search list Filter

Group

	Type	Key	Summary	Status	Comments	Sprint	Assignee	
☐		ESS-58	Implement room and seating plan creation	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-59	Store seating layout in database	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-60	Implement check-in workflow	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-61	Implement photo upload	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-62	Call ML verification service	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-63	Validate seat compliance	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-64	Save check-in record	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-69	Implement reporting features	IN PROGRESS	Add comment		Burçak Çelt	
☒		ESS-70	List check-in results	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-71	List mismatches and violations	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-72	Filter reports by exam	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-73	ML Integration	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-74	Implement ML wrapper service	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-76	Return match / no match result	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-77	Handle invalid image cases	IN PROGRESS	Add comment		Burçak Çelt	

+ Create

Quickstart

burcakcelt.atlassian.net

Jira

Search

+ Create Upgrade

Spaces

Exam Security System

Summary Backlog Board Timeline List Forms Development Pages

Search list Filter

Group

	Type	Key	Summary	Status	Comments	Sprint	Assignee	
☐		ESS-73	ML Integration	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-74	Implement ML wrapper service	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-76	Return match / no match result	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-77	Handle invalid image cases	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-75	Integrate face verification library	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-78	Testing & Validation	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-79	Prepare test cases document	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-80	Write functional test cases	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-81	Write negative test cases	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-82	Write edge case scenarios	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-83	Implement unit tests	IN PROGRESS	Add comment		Ecem Özer	
☐		ESS-84	Test seating compliance logic	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-85	Documentation & Finalization	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-86	Write README file	IN PROGRESS	Add comment		Burçak Çelt	
☐		ESS-87	Final review and cleanup	IN PROGRESS	Add comment		Burçak Çelt	

+ Create

Quickstart

Database Design and Implementation

The database layer of the Exam Security System was designed to support identity verification, seating compliance, and violation tracking in a consistent and reliable manner. The schema was created based on the ERD and business rules defined in the analysis phase. Referential integrity and validation constraints were used to enforce critical system rules such as role-based access, seat assignment consistency, and prevention of duplicate check-ins.

Database Schema Design

The database consists of several core entities that represent users, students, exams, rooms, seating plans, check-in records, and violations. Relationships between entities were established using foreign keys to ensure data consistency.

Users are stored with role information to support authentication and authorization mechanisms. Students are stored separately with their identification information and registered ID photos. Exams are linked to rooms, and each room contains multiple seats defined by row and column information. Seating assignments link students to expected seats for a specific exam, enabling seat compliance validation during check-in.

The check-ins table records the result of each student's exam entry, including the identity verification outcome, seat compliance result, overall status, and timestamp. Violations are stored as separate records linked to a specific check-in, allowing detailed incident tracking without duplicating check-in data.

Database Constraints and Business Rules

Several constraints were applied at the database level to support business rules. Each student is allowed to check in only once per exam, enforced using a unique constraint on the exam and student combination in the check-ins table. Seating assignments are unique per exam to prevent

multiple students from being assigned to the same seat. Foreign key constraints ensure that all check-ins and violations reference valid users, students, exams, and seats.

These constraints reduce the likelihood of inconsistent data and support reliable testing and validation of system behavior.

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL,  
    password_hash TEXT NOT NULL,  
    role TEXT NOT NULL CHECK (role IN ('ADMIN', 'PROCTOR')),  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE students (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_no TEXT UNIQUE NOT NULL,  
    full_name TEXT NOT NULL,  
    id_photo_path TEXT,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE rooms (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    rows INTEGER NOT NULL,
```

```
cols INTEGER NOT NULL,  
  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE exams (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    title TEXT NOT NULL,  
  
    exam_datetime DATETIME NOT NULL,  
  
    room_id INTEGER NOT NULL,  
  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (room_id) REFERENCES rooms(id)  
);
```

```
CREATE TABLE seats (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    room_id INTEGER NOT NULL,  
  
    seat_code TEXT NOT NULL,  
  
    row_no INTEGER NOT NULL,  
  
    col_no INTEGER NOT NULL,  
  
    FOREIGN KEY (room_id) REFERENCES rooms(id)  
);
```

```
CREATE TABLE seating_assignments (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    exam_id INTEGER NOT NULL,
```

```
student_id INTEGER NOT NULL,  
  
seat_id INTEGER NOT NULL,  
  
assigned_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
UNIQUE (exam_id, student_id),  
  
UNIQUE (exam_id, seat_id),  
  
FOREIGN KEY (exam_id) REFERENCES exams(id),  
  
FOREIGN KEY (student_id) REFERENCES students(id),  
  
FOREIGN KEY (seat_id) REFERENCES seats(id)  
  
);
```

```
CREATE TABLE checkins (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    exam_id INTEGER NOT NULL,  
  
    student_id INTEGER NOT NULL,  
  
    proctor_id INTEGER NOT NULL,  
  
    expected_seat_id INTEGER,  
  
    actual_seat_id INTEGER,  
  
    photo_path TEXT,  
  
    ml_result TEXT CHECK (ml_result IN ('MATCH', 'NO_MATCH')),  
  
    seat_result TEXT CHECK (seat_result IN ('CORRECT', 'WRONG', 'UNKNOWN')),  
  
    status TEXT CHECK (status IN ('APPROVED', 'FLAGGED', 'REJECTED')),  
  
    checked_in_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
    UNIQUE (exam_id, student_id),  
  
    FOREIGN KEY (exam_id) REFERENCES exams(id),  
  
    FOREIGN KEY (student_id) REFERENCES students(id),
```

```
FOREIGN KEY (proctor_id) REFERENCES users(id),  
FOREIGN KEY (expected_seat_id) REFERENCES seats(id),  
FOREIGN KEY (actual_seat_id) REFERENCES seats(id)  
);
```

```
CREATE TABLE violations (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    exam_id INTEGER NOT NULL,  
    student_id INTEGER NOT NULL,  
    proctor_id INTEGER NOT NULL,  
    checkin_id INTEGER NOT NULL,  
    reason TEXT NOT NULL,  
    notes TEXT,  
    evidence_path TEXT,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (exam_id) REFERENCES exams(id),  
    FOREIGN KEY (student_id) REFERENCES students(id),  
    FOREIGN KEY (proctor_id) REFERENCES users(id),  
    FOREIGN KEY (checkin_id) REFERENCES checkins(id)  
);
```

Dummy Data Usage

To support development, testing, and demonstration of system functionality, dummy data was inserted into the database. Sample admin and proctor users were created to test role-based

access control. Example rooms, exams, and seating plans were defined to simulate a real exam environment. Sample student records and seating assignments were added to validate seat compliance logic during the check-in process. Dummy check-in and violation records were also used to test reporting and incident tracking features.

Testing and Validation

Testing and validation activities were conducted to ensure that the Exam Security System behaves correctly under normal and abnormal conditions. The testing approach includes functional testing, negative testing, and edge-case testing for critical workflows such as authentication, check-in operations, seating compliance validation, and violation logging. In addition, unit tests were implemented for key business logic and validation rules, including duplicate check-in prevention, role-based permission enforcement, seating compliance logic, and the ML verification wrapper service using mock inputs. The ML model itself was not unit-tested; instead, the wrapper/service that the system calls was tested to validate correct integration and decision handling.

Test Cases Document

The following test cases were prepared to cover the core requirements and business rules of the system. Each test case includes preconditions, steps, inputs, and expected results to provide clear validation criteria.

Functional Test Cases

TC-01	–	Proctor	Login	(Valid	Credentials)
Precondition:	Proctor	account	exists	in	the system.
Steps:	Open login page	→ enter valid username/password	→ click login.		
Input:	Valid	proctor	credentials.		
Expected Result:	Login succeeds and proctor dashboard is displayed.				
TC-02	–	Admin	Creates	Exam	(Valid Data)
Precondition:	Admin	is	logged in.	A	room exists.
Steps:	Navigate to exam management	→ enter title/date/time	→ select room	→ save.	

Input: Valid exam information.
Expected Result: Exam is created and appears in exam list.

TC-03 – Seating Plan Creation (Row/Column Structure)

Precondition: Admin is logged in. Room exists with defined rows/cols.

Steps: Navigate to seating plan → generate seats → save plan.

Input: Room rows/cols (e.g., 5x6).

Expected Result: Seats are created (e.g., A1...E6) and stored for the room/exam.

TC-04 – Successful Check-in (MATCH + Correct Seat)

Precondition: Proctor is logged in. Student is assigned a seat for the exam. Student has a registered ID photo.

Steps: Select exam/student → upload live photo → enter actual seat code (same as expected) → submit.

Input: Valid live photo, correct seat code.

Expected Result: ML result = MATCH, seat result = CORRECT, status = APPROVED, check-in saved with timestamp.

TC-05 – Violation Logging After Flagged Check-in

Precondition: A check-in is completed with status = FLAGGED.

Steps: Open violation form → enter reason → optionally enter notes/evidence → submit.

Input: Reason = "Identity mismatch" (example).

Expected Result: Violation is saved and linked to the related check-in; confirmation is shown.

TC-06 – Reporting: View Check-ins and Violations

Precondition: At least one check-in and one violation exist. Authorized user is logged in.

Steps: Open reports page → filter by exam → view results.

Input: Exam filter selection.

Expected Result: System lists check-ins (approved/flagged) and violation records correctly.

Negative Test Cases

TC-07 – Login with Invalid Password

Precondition: User account exists.

Steps: Enter correct username with incorrect password → login.

Input: Wrong password.

Expected Result: Login fails; user is not authenticated; error message displayed.

TC-08 – Duplicate Check-in Attempt (Same Student + Exam)

Precondition: Student already checked in for the exam.

Steps: Try to perform check-in again for same student and exam.

Input: Same student_id and exam_id.

Expected Result: System rejects the attempt and displays “Duplicate check-in not allowed.”
No new record is created.

TC-09 – Unauthorized Access to Admin Features
Precondition: Proctor is logged in.
Steps: Attempt to access exam creation / seating plan management pages.
Input: Direct URL access or navigation attempt.
Expected Result: Access denied (403 or redirected). Admin functions are not accessible to proctor.

TC-10 – Violation Submission Without Reason
Precondition: Proctor opens violation form.
Steps: Leave reason empty → submit.
Input: reason = empty.
Expected Result: Validation error is shown; violation is not saved.

Edge Case Test Cases

TC-11 – Missing Image in Check-in
Precondition: Proctor is logged in and check-in started.
Steps: Attempt to submit check-in without uploading a photo.
Input: No image uploaded.
Expected Result: System blocks submission and shows “Photo is required.”

TC-12 – Invalid Image Format
Precondition: Proctor is logged in.
Steps: Upload a non-image file (e.g., .txt) as live photo.
Input: Invalid file type.
Expected Result: System rejects file and shows format error.

TC-13 – ML Returns NO_MATCH (Identity Mismatch)
Precondition: Student has registered ID photo.
Steps: Upload a different person’s photo as live photo.
Input: Live photo that should not match.
Expected Result: ML result = NO_MATCH; status becomes FLAGGED; system prompts for violation logging.

TC-14 – Seat Mismatch (Wrong Seat)
Precondition: Student has an expected seat assigned.
Steps: Enter a seat code different from expected.
Input: actual_seat_code != expected_seat_code.
Expected Result: seat_result = WRONG; status becomes FLAGGED; check-in saved with mismatch status.

TC-15 – Multiple Faces Detected (If Supported by Library)
Precondition: ML wrapper supports multi-face detection.
Steps: Upload a photo containing multiple faces.
Input: Multi-face image.
Expected Result: System flags the check-in and prevents auto-approval; status = FLAGGED (or REJECTED depending on rule).

Unit Tests

Unit tests were implemented to validate critical business logic without relying on UI behavior. The following components were covered:

The seating compliance logic was unit-tested by comparing expected and actual seat codes and verifying that the correct compliance result is returned for correct, wrong, and missing seat input cases.

Validation rules were unit-tested to ensure required fields are enforced and that duplicate check-ins are prevented using the exam and student identifiers. Role permission checks were also unit-tested to confirm that proctors cannot access admin-only actions.

The ML integration wrapper service was unit-tested using mock inputs. The goal was to verify that the system correctly interprets the wrapper output (MATCH / NO_MATCH) and handles invalid input cases, without attempting to test the internal ML model.

Web Screens

**maltepe** üniversitesi
İSTANBUL 1997

Exam Security System



Login

Username

Password

Login

Access for Proctor and Admin roles.

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Welcome, **admin!**  Logout 

 Exams  Rooms  Seating Plans  Roster  Admin

Admin Dashboard

**Exams**
Manage all scheduled exams.

**Rooms**
Configure and manage rooms.

**Seating Plans**
Assign seats for exams.

**Roster**
View and edit student roster.

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Welcome, **proctor!** 

 Check-in  Reports  Help

Proctor Dashboard

**Check-in**
Start and manage student check-ins.

**Reports**
View detailed check-in reports.

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Welcome, **admin!**  Logout 

 Exams  Rooms  Seating Plans  Roster  Admin

 Admin / Exams > Exam Management

Create Exam

 Exams

 Rooms

 Seating Plans

 Roster

 Admin

Title	Date / Time	Room	Status
Math 101 Midterm	Aug 1, 2024 · 10:00	Room A101	Scheduled  
History 210 Final	Aug 10, 2024 · 13:00	Room B201	Scheduled  
Physics 102 Midterm	Aug 12, 2024 · 9:00	Room A202	Scheduled  
Chemistry 220 Final	Jul 25, 2024 · 15:00	Room A103	Completed  

Showing 1 to 4 of 4 items.  

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Check-in  Reports  Help

 Check-in

 Reports

 Help

Proctor Dashboard

 Check-in
Start and manage student check-ins.

Create Exam

Title

Enter exam title...

Room

Room A101

2024-08-15T10:30

August

1 23 64 32 10 30

4

AM

10 : 30 AM

11 : 30 AM

Cancel 

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Welcome, **admin!**  Logout 

 Exams  Rooms  Seating Plans  Roster  Admin

 Admin / Rooms > Room Management

 Exams

 Rooms

 Seating Plans

 Roster

 Admin

Room Name	Rows	Columns	Status
Room A101	4	6	 
Room B201	5	7	 
Room A202	6	6	 
Room A103	4	5	 

Showing 1 to 4 of 4 items.  

Maltepe University Exam Security System © 2024.

**maltepe** üniversitesi
İSTANBUL 1997

Check-in  Reports  Help

 Check-in

 Reports

 Help

Room Management

Room Name	Rows	Columns
Room A101	4	6
Room B201	5	7
Room A202	6	6

Add Room

Name

Room C301

Rows

5

Columns

8

Cancel 

Maltepe University Exam Security System © 2024.

maltepe üniversitesi
İSTANBUL 1997

Welcome, admin! Logout

ExamsRoomsSeating PlansRosterAdmin

Admin / Seating Plans > Seating Plan Builder

Room: Room C301 (5 rows, 8 columns)

A1	A2	A3	A4	A5	A6	A7	A8	A9
B1	B8	B3	B4	B5	K0	C7	C8	B9
C1	C2	C3	C4	C5	C8	C7	C8	C8
D1	D2	D3	D4	D5	D6	D7	D8	E9
E1	E2	E3	E4	E5	E6	E7	E8	E8

< BackSave Seating Plan

Maltepe University Exam Security System © 2024.

maltepe üniversitesi
İSTANBUL 1997

Welcome, admin! Logout

ExamsRoomsSeating PlansRosterAdmin

Admin / Roster > Student Roster > Seating Plan Builder

Student Roster

Import CSVAdd Student

Student No	Full Name	ID Photo	
220706038	Ecem Nur Özer	 Yes	EditDelete
220706034	Burçak Çelt	 Yes	EditDelete

Showing 1 to 2 of 2 items. < Previous 10 Next >

Maltepe University Exam Security System © 2024.

maltepe üniversitesi
İSTANBUL 1997

Welcome, admin! Logout

ExamsRoomsRosterReports

Student Roster

Student No	Full Name	ID Photo
220706038	Ecem Nur Özer	Yes
220706034	Burçak Celt	No

Showing 1 to 2 of 2 items.

Add Student

Student No
220706040

Full Name
Buşra Yılmaz

ID Photo


Choose File busyia.jpg

CancelAdd Student

Maltepe University Exam Security System © 2024.

Student Check-in

Exam:

Math 101 Midterm - Room A101 | Aug 1, 2024 10:00

Student

Ecem Nur Özer , 220706038

Capture Photo:



Capture Photo

Upload Photo

< Back

Match Result:

MATCH

NO_MATCH

Expected Seat:

A3

A3

Actual Seat:

A3

Submit Check-in

Checked in: Aug 1, 2024 10:11 AM

Maltepe University Exam Security System © 2024.

Student Check-in

Face match and seat confirmed.

APPROVED

Log Violation

Check-in Result



Face match and seat confirmed.

Reasons:

- Face mismatch
- Wrong seat selected
- Multiple faces detected
- Missing ID photo

Go to Dashboard

Checked in: Aug 1, 2024 10:11 AM

Log Violation

Maltepe University Exam Security System © 2024.

Check-in Result

Face match and seat confirmed.

Log Violation

Maltepe University Exam Security System © 2024.

maltepe üniversitesi
ESTABLISHED 1972

Welcome, **proctor!**  Logout 

 Check-in  Reports  Help

Event Logs

Date filter:

Jul 25, 2024 - Aug 1, 2024 - Custom date range 

07/21/2024 

Filter

Role:

All

Proctor

Student Name:

Date	Role		Event	Details
Aug 1 2024 10:11 AM	Proctor	Plontor Proctor	Check-in Approved	Ecem Nur
Aug 1 2024 9:15 AM	Proctor	Proctor Proctor	Check-in Approved	Ecem Nur
Jul 25 2024 9:15 AM	Proctor	Crocttr	Exam Created	Math 101 M

Showing 1 to 3 of 3 items.

Event Details

Date/Time: Aug 1, 2024 10:11 AM

Role: Proctor

Event: Check-in Approved

Details: Ecem Nur Ozer
Student No: 220706038
Expected Seat, A3
Actual Seat, A3

Close

Maltepe University Exam Security System © 2024.

maltepe üniversitesi
ESTABLISHED 1972

Welcome, **proctor!**  Logout 

 Check-in  Reports  Help

Help & Support

Frequently Asked Questions

How do I start the student check-in process? 

Navigate to the Check-in page, select the exam and the student, capture or upload their photo, and confirm the seat assignment. Click "Submit Check-in" to complete the process num.

What should I do if a student is seated incorrectly? 

How can I manage exam seating plans? 

How do I view the event log for check-in activities? 

What should I do if I can't recognize a student's ID photo? 

How can I add new students to the roster? 

Maltepe University Exam Security System © 2024.

Dashboard

Welcome back! Here is an overview of recent activities and data for the exam security system.

4

Upcoming Exams

58

Total Check-ins

2

Issues Logged

94

Registered Students

Upcoming Exams

Exam	Date & Time	Room	Status
Math 101 Midterm Room A101	Aug 1, 2024 10:00	Room A101	Finalized
CS 201 Final Room B202	Aug 5, 2024 13:00	Room B202	In Progress

Manage Exams

Recent Event Logs

Date	Event	Details
Aug 1, 2024 11:01 AM	Issue Logged	Büşra Yılmaz, Room B202
Aug 1, 2024 10:11 AM	Check-in Approved	Ecem Nur Özer, Room A101 Expected: A3
Jul 25, 2024 9:15 AM	Admin Created	Math 101 Midterm Room A101

View All Logs

Check-in / Log Violation

Log Violation



Büşra Yılmaz, Room B202
Expected Seat: B5

Reason

Seat mismatch

Notes

Student was sitting in seat A8 instead of assigned seat B5.

Evidence



mismatch_photo.jpg

Choose File

Save Violation



Access Denied

You do not have permission to access this page.

This section is restricted to admin users only.

[Back to Dashboard](#)



